



INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO®
UNIDAD DE EDUCACIÓN DIGITAL

DDL (Lenguaje de Definición de Datos)

Autor: Oralia del Socorro Cortés Grajales



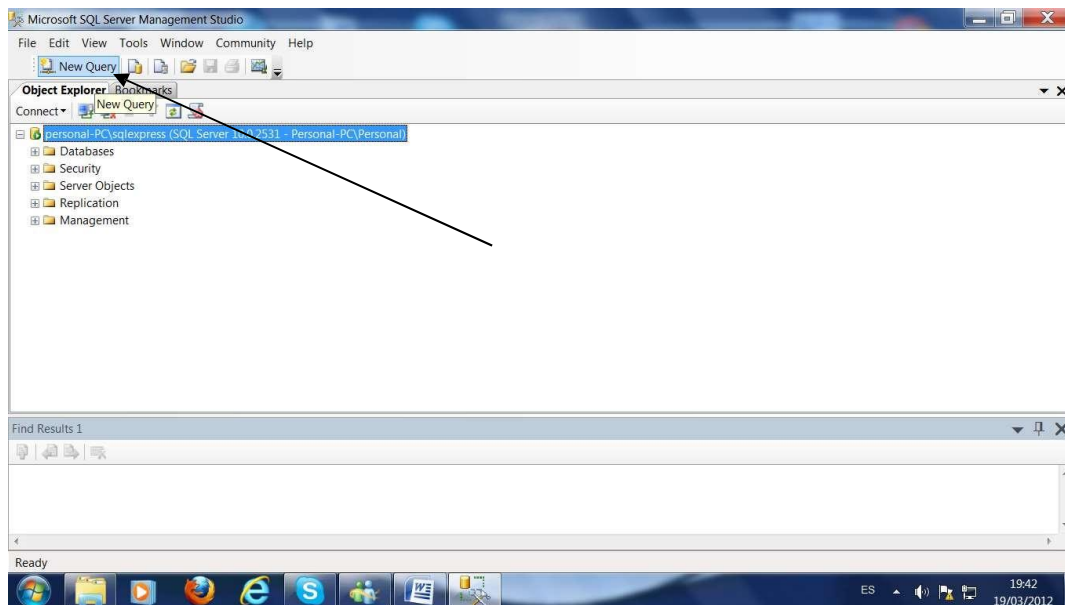
DDL (Lenguaje de Definición de Datos)

Una vez se tenga instalado el ambiente de trabajo pueden implementar las bases de datos con sus relaciones utilizando los comandos DDL (Lenguaje de definición de datos).

DDL

Comando	Descripción
CREATE	Utilizado para crear objetos como nuevas bases de datos, tablas, campos e índices
DROP	Empleado para eliminar objetos como tablas e índices, vistas.
ALTER	Utilizado para modificar objetos como las tablas agregando campos o cambiando la definición de los campos.

Estando en el ambiente de trabajo SQL server en New Query le da doble clic apareciendo un editor donde puede escribir los siguientes comandos:



Lo que está entre /* */ son comentarios

/* Crear la base de datos*/

create database Construcccion

/******solo ejecuta estas dos lineas******/

/* activa la base de datos construcccion*/

/******solo ejecuta esta linea******/

use Construcccion

/* Crear tabla oficio mirar el comando abajo*/

create table tblOficio(Tipo_Oficio **varchar**(30),

Bonificacion **int**,

check(Bonificacion >= 30000 and Bonificacion <= 80000),

Horas_Semana **int**,

check(Horas_Semana >= 30 and Horas_Semana <= 60),

Primary key (Tipo_Oficio))

/***Selecciona todo el bloque de la tabla y ejecuta***/

/*** nota si la tabla le queda mal construida con este comando la puede borrar luego la arregla y la vuelve a ejecutar*****/

/*Crear tabla Trabajador*/

create table tblTrabajador(Cedula **int**,

Nombre **varchar**(50) not null,

Valor_Hora **int**,

/*check es el comando que se utiliza para hacer las validaciones*/

check(Valor_Hora >= 7000 and Valor_Hora <= 20000),

Tipo_Oficio **varchar**(30),

primary key (Cedula),

foreign key (Tipo_Oficio)**references**

tblOficio(Tipo_Oficio))

/*Crear tabla Area*/

create table tblArea(Cod_Area **varchar**(10),

Nom_Area **Varchar**(30),

Estrato **int**,

```
check(Estrato >= 1 and Estrato <= 7),  
primary key (Cod_Area))
```

```
/*Crear tabla edificio*/  
create table tblEdificio(Iden_Edif int,  
Direccion varchar(30),  
Tipo varchar (30),  
Calidad int,  
check (Calidad >= 1 and Calidad <= 5),  
Categoria int,  
check(Categoria >= 1 and Calidad <= 5),  
Cod_Area varchar (10)  
primary key (Iden_Edif),  
foreign key(Cod_Area)references tblArea(Cod_Area))
```

```
/*Crear tabla Asignar*/  
create table tblAsignar(Cedula int,  
Iden_Edif int,  
Fecha_I datetime,  
Num_Dias int,  
check (Num_Dias > 0),  
Primary key(Cedula,Iden_edif)
```

```
foreign key(Cedula) references tbltrabajador(cedula), foreign  
key(Iden_edif) references tblEdificio(Iden_Edif))
```

/* insertamos datos en cada tabla, los datos deben corresponder con el orden dado en la definición*/

```
Insert into tbloficio values('Decorador',30000,35)  
Insert into tbloficio values('Albañil',35000,37)  
Insert into tbloficio values('Carpintero',31000,40)  
Insert into tbloficio values('Electrico',35000,35)  
Insert into tbloficio values('Arquitecto',40000,30)
```

```
/**Para ver los datos de oficio seleccionamos y ejecutamos*****/ Select *  
from tbloficio
```

/*****insertar registros para trabajador*****/
/*sugerencia ejecute por linea para que reconozca facilmente el error si lo hay**/

```
into tbltrabajador values(1235,'Annie',12500,'Electrico')
Insert into tbltrabajador values(1311,'Roberto',15750,'Decorador')
Insert into tbltrabajador values(1412,'Carlos',13700,'Decorador')
Insert into tbltrabajador values(1415,'Lina',12500,'Decorador')
Insert into tbltrabajador values(1418,'Pedro',10000,'Carpintero')
Insert into tbltrabajador values(1520,'Luis',11750,'Electrico')
Insert into tbltrabajador values(1525,'Juan',20000,'Arquitecto')
Insert into tbltrabajador values(2920,'Raul',10000,'Carpintero')
Insert into tbltrabajador values(3001,'Gabriel',15500,'Decorador')
Insert into tbltrabajador values(3231,'Alvaro',17400,'Decorador')
Insert into tbltrabajador values(4446,'Mario',8200,'Albañil')
Insert into tbltrabajador values(8520,'Bernardo',8500,'Albañil')
```

```
Insert into tblarea values('a10','Sur',2)
Insert into tblarea values('a11','Sur',4)
Insert into tblarea values('a12','Norte',3)
Insert into tblarea values('a13','Norte',2)
```

```
Insert into tbledificio values(111,'calle1213','Oficina',3,2,'a10')
Insert into tbledificio values(210,'calle1222','Oficina',4,3,'a10')
Insert into tbledificio values(215,'calle1215','Comercio',5,3,'a11')
Insert into tbledificio values(312,'calle1313','Oficina',5,2,'a13')
Insert into tbledificio values(435,'calle1513','Comercio',2,4,'a13')
Insert into tbledificio values(460,'calle1263','Almacen',4,4,'a12')
Insert into tbledificio values(520,'calle1223','Residencia',3,5,'a10')
Insert into tbledificio values(820,'calle1245','Almacen',3,5,'a10')
```

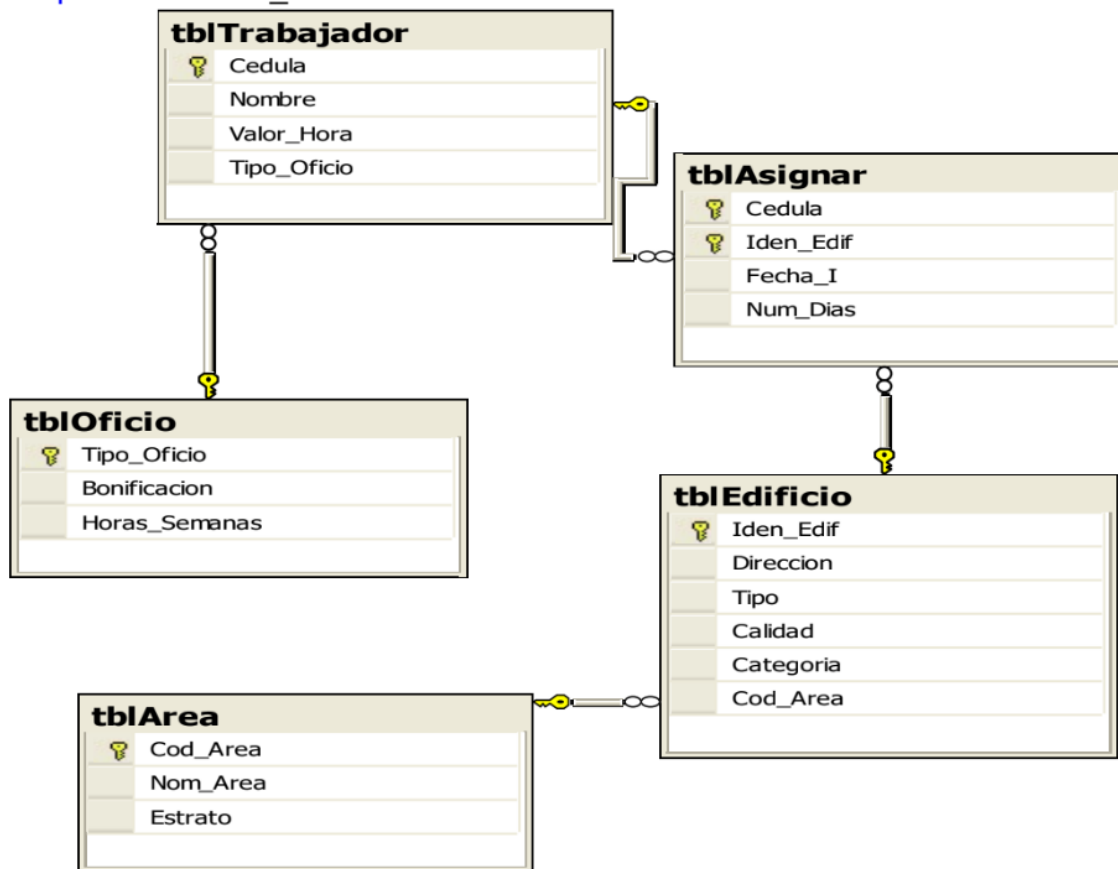
```
Insert into tblasignar values(1235,111,'02/02/02',25)
Insert into tblasignar values(1235,210,'01/03/02',5)
Insert into tblasignar values(1235,520,'02/02/02',25)
Insert into tblasignar values(1235,820,'01/03/02',5)
```

```
Insert into tblasignar values(1235,215,'02/07/02',10)
Insert into tblasignar values(1235,312,'02/02/02',10)
Insert into tblasignar values(1235,435,'01/02/04',22)
Insert into tblasignar values(1235,460,'02/02/05',21)
Insert into tblasignar values(1412,210,'02/02/02',22)
Insert into tblasignar values(1412,312,'02/02/02',14)
Insert into tblasignar values(1418,435,'02/02/02',23)
Insert into tblasignar values(1418,460,'02/02/02',10)
Insert into tblasignar values(3231,312,'02/02/02',25)
Insert into tblasignar values(3231,111,'02/02/02',13)
Insert into tblasignar values(4446,111,'02/02/02',18)
```

Este es el diagrama correspondiente a la base de datos que acabamos de crear.

/* borrar una tabla cuando se equivoca en la definición */

Drop table nombre_tabla



Comandos para modificar y borrar objetos de la base de datos

Se crea una base de datos como se muestra a continuación.

create database modifica
use modifica

```

create table empleado
(
    ced int ,
    nom varchar(10),
    salario int
)
    
```

```
create table tarea
(
cod_t int ,
nom_t varchar(50),
valor_t int
)
```

Alterar tabla

Con los comandos de alterar la tabla se le realizara las siguientes modificaciones

```
/*adicionar campo
...adicionar el campo telefono para empleado
*/
```

```
alter table empleado add telefono int not null
```

```
/*modificar campo
...cambiar el tamaño del campo nombre del empleado por 50
*/
```

```
alter table empleado alter column nom varchar(50) not null
```

```
/*eliminar campo
...eliminar el campo valor_t
*/
```

```
alter table tarea drop column valor_t
```

```
/*adicionar clave principal
...modifico campo para q no acepte null y adiciono la clave principal
*/
```

```
alter table empleado alter column ced int
not null alter table empleado add primary
key (ced)
/*adicionar clave principal para tarea*/
```



```
alter table tarea alter column cod_t int not null
```

```
alter table tarea add primary key (cod_t)
```

```
/*adicionar clave foranea  
...adicionar clave foranea ced en tarea  
*/
```

```
alter table tarea add ced int not null
```

```
alter table tarea add foreign key (ced) references  
empleado(ced) ON delete cascade ON update cascade
```

```
/*adicionar regla de validacion  
...validar q salario entre 500000 y 1500000  
*/
```

```
alter table empleado add constraint ck_salario check  
(salario>=500000 and salario<=1500000)
```

Borrar tabla

```
Drop table nombretabla
```

```
/******Crear indices*****/  
/*Crear índice único*/
```

```
CREATE UNIQUE INDEX nom_indce ON nom_tabla(campo)
```

```
/*Crear índice con duplicado*/
```

```
CREATE INDEX nom_indce ON nom_tabla(campo)
```

```
/*borrar indice*/
```

```
DROP INDEX nom_tabla. Nom_indice
```

Reglas de integridad (RI)

Para realizar las reglas de integridad que se vieron en la unidad I debe tener en cuenta los siguientes operadores

OPERADORES LÓGICOS

Operador	Uso
AND	Es el "y" lógico. Evalúa dos condiciones y devuelve un valor de verdad sólo si ambas son ciertas.
OR	Es el "o" lógico. Evalúa dos condiciones y devuelve un valor de verdadero si alguna de las dos es cierta.
NOT	Negación lógica. Devuelve el valor contrario de la expresión.

OPERADORES DE COMPARACIÓN

Operador	Uso
<	Menor que
>	Mayor que
<>	Distinto que
<=	Menor o igual que
>=	Mayor o igual que
=	Igual que
BETWEEN	Utilizado para especificar un intervalo de valores
LIKE	Utilizado en la comparación de un modelo
In	Utilizado para especificar registros de una base de datos

Las RI son:1

1.Semánticas

En el mundo real existen ciertas restricciones que deben cumplir los elementos en él existentes; por ejemplo, una persona sólo puede tener un número de DNI y una única dirección oficial. Cuando se diseña una base de datos se debe reflejar fielmente el universo del discurso que estamos tratando, lo que es lo mismo, reflejar las restricciones existentes en el mundo real. Tienen que ser definidas por los diseñadores en el modelo entidad relación.

2. Reglas de negocio

Los usuarios o los administradores de la base de datos pueden imponer ciertas restricciones específicas sobre los datos, denominadas reglas de negocio.

Por ejemplo, si en una oficina de la empresa inmobiliaria sólo puede haber hasta veinte empleados, el SGBD debe dar la posibilidad al usuario de definir una regla al respecto y debe hacerla respetar. En este caso, no debería permitir dar de alta un empleado en una oficina que ya tiene los veinte permitidos.

Integridad de dominio: restringimos los valores que puede tomar un atributo respecto a su dominio, por ejemplo $EDAD \geq 18$ and $EDAD \leq 65$. Una nota ≥ 0 and nota ≤ 5

3. Reglas de la entidad

- **Tipos de datos:** Al momento de definir la tabla se deben definir correctamente los tipos de datos de los atributos o columnas.
- **Permitir valores NULL**

1 Reglas de integridad. <https://docs.microsoft.com/es-es/previous-versions/sql/sql-server-2008/r2/ms190765%28v%3dsql.105%29>

La nulabilidad de una columna determina si las filas de una tabla pueden contener un valor NULL en esa columna. Un valor NULL no es lo mismo que cero (0), en blanco o que una cadena de caracteres de longitud cero, como "". NULL significa que no hay ninguna entrada. La presencia de un valor NULL suele implicar que el valor es desconocido o no está definido. Por ejemplo, un valor NULL en la columna fecha_venta de la tabla Producto no implica que el artículo no tenga una fecha de venta final. El valor NULL significa que se desconoce la fecha o que no se ha establecido.

- **Restricciones UNIQUE**

Puede utilizar restricciones UNIQUE para garantizar que no se escriben valores duplicados en columnas específicas que no forman parte de una clave principal. Tanto la restricción UNIQUE como la restricción PRIMARY KEY exigen la unicidad; sin embargo, debe utilizar la restricción UNIQUE y no PRIMARY KEY si desea exigir la unicidad de una columna o una combinación de columnas que no forman la clave principal.

En una tabla se pueden definir varias restricciones UNIQUE, pero sólo una restricción PRIMARY KEY.

Además, a diferencia de las restricciones PRIMARY KEY, las restricciones UNIQUE admiten valores NULL. Sin embargo, de la misma forma que cualquier valor incluido en una restricción UNIQUE, sólo se admite un valor NULL por columna.

Es posible hacer referencia a una restricción UNIQUE con una restricción FOREIGN KEY.

- **Creación de índices**

Los índices son "estructuras" alternativa a la organización de los datos en una tabla. El propósito de los índices es acelerar el acceso a los datos mediante operaciones físicas más rápidas y efectivas. Para entender mejor la importancia de un índice pongamos un ejemplo; imagínate que tienes delante las páginas amarillas, y deseas buscar el teléfono de Manuel Salazar que vive en Alicante. Lo que harás será buscar en ese pesado libro la población Alicante, y guiándote por la cabecera de las páginas buscarás los apellidos que empiezan por S de Salazar. De esa forma localizarás más rápido el apellido Salazar. Pues bien, enhorabuena, has estado usando un índice.

Pues el objetivo de definir índices en SQL Server es exactamente para conseguir el mismo objetivo: acceder más rápido a los datos.

Consideraciones para usar índices

Columnas selectivas

Columnas afectadas en consultas de rangos: BETWEEN, mayor que, menor que, etc.

Columnas accedidas "secuencialmente" Columnas implicadas en JOIN, GROUP BY Acceso muy rápido a filas: lookups

Ejemplo:

/*Crear índice único*/

```
CREATE UNIQUE INDEX nom_indce ON nom_tabla(campo)
```

/*Crear índice con duplicado*/

```
CREATE INDEX nom_indce ON nom_tabla(campo)
```

/*borrar índice*/

```
DROP INDEX nom_tabla. Nom_indice
```

- **Definiciones DEFAULT**

Cada columna de un registro debe contener un valor, aunque sea un valor NULL. Puede haber situaciones en las que deba cargar una fila de datos en una tabla, pero no conozca el valor de una columna o el valor ya no exista. Si la columna acepta valores NULL, puede cargar la fila con un valor NULL. Pero, dado que puede no resultar conveniente utilizar columnas que acepten valores NULL, una mejor solución podría ser establecer una definición DEFAULT para la columna siempre que sea necesario. Por ejemplo, es habitual especificar el valor cero como valor predeterminado para las columnas numéricas, o N/D (no disponible) como valor predeterminado para las columnas de cadenas cuando no se especifica ningún valor.

Ejemplo

fecha datetime default getdate() indica que el valor predeterminado de fecha es la fecha actual del sistema.

4. Regla de integridad referencial

Son reglas de integridad controladas por el DBGS

▮ Restricciones PRIMARY KEY

Una tabla suele tener una columna o una combinación de columnas cuyos valores identifican de forma única cada fila de la tabla. Estas columnas se denominan claves principales de la tabla y exigen la integridad de entidad de la tabla. Puede crear una clave principal mediante la definición de una restricción PRIMARY KEY cuando cree o modifique una tabla.

Una tabla sólo puede tener una restricción PRIMARY KEY y ninguna columna a la que se aplique una restricción PRIMARY KEY puede aceptar valores NULL. Debido a que las restricciones PRIMARY KEY garantizan datos únicos, con frecuencia se definen en una columna de identidad.

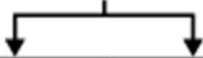
Cuando especifica una restricción PRIMARY KEY en una tabla, Database Engine (Motor de base de datos) exige la unicidad de los datos mediante la creación de un índice único para las columnas de clave principal. Este índice también permite un acceso rápido a los datos cuando se utiliza la clave principal en las consultas. De esta forma, las claves principales que se eligen deben seguir las reglas para crear índices únicos.

Si se define una restricción PRIMARY KEY para más de una columna, puede haber valores duplicados dentro de la misma columna, pero

cada combinación de valores de todas las columnas de la definición de la restricción PRIMARY KEY debe ser única.

Como se muestra en la siguiente ilustración, las columnas ProductID y VendorID de la tabla ProductVendor forman una restricción PRIMARY KEY compuesta para esta tabla. Así se garantiza que la combinación de ProductID y VendorID es única.

Clave principal



ProductID	VendorID	AverageLeadTime	StandardPrice	LastReceiptCost
1	1	17	47.8700	50.2635
2	104	19	39.9200	41.9160
7	4	17	54.3100	57.0255
609	7	17	25.7700	27.0585
609	100	19	28.1700	29.5785

Tabla ProductVendor

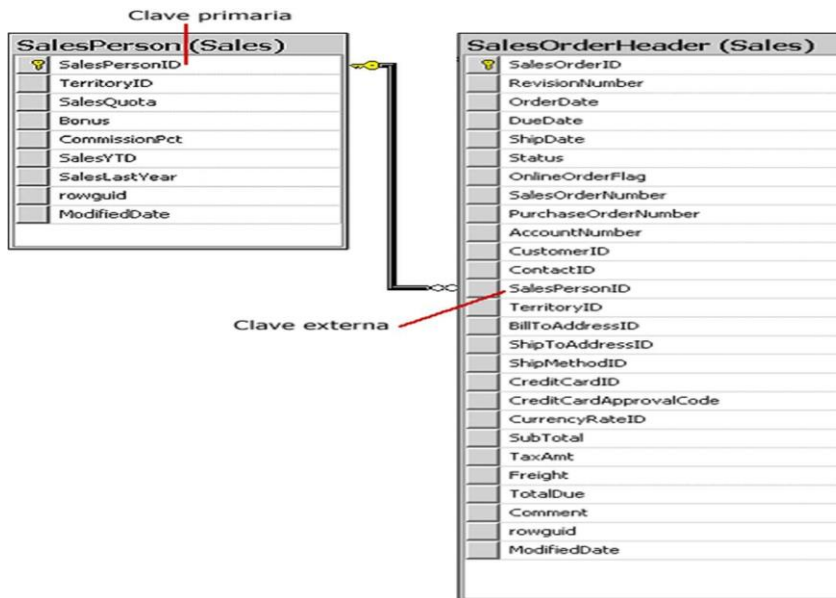
Cuando trabaja con combinaciones, las restricciones PRIMARY KEY relacionan una tabla con otra. Por ejemplo, para determinar los proveedores que suministran determinados productos, puede utilizar una combinación de tres elementos entre las tablas Vendor, Production.Product y ProductVendor. Puesto que ProductVendor contiene las columnas de ProductID y VendorID, se puede obtener acceso a las tablas Product y Vendor mediante su relación con ProductVendor.

▮ Restricciones FOREIGN KEY

Una clave externa (FK) es una columna o combinación de columnas que se utiliza para establecer y exigir un vínculo entre los datos de dos tablas. Puede crear una clave externa mediante la definición de una restricción FOREIGN KEY cuando cree o modifique una tabla.

En una referencia de clave externa, se crea un vínculo entre dos tablas cuando las columnas de una de ellas hacen referencia a las columnas de la otra que contienen el valor de clave principal. Esta columna se convierte en una clave externa para la segunda tabla.

Por ejemplo, la tabla Sales.SalesOrderHeader de la base de datos AdventureWorks tiene un vínculo a la tabla Sales.SalesPerson porque existe una relación lógica entre pedidos de ventas y personal de ventas. La columna SalesPersonID de la tabla SalesOrderHeader coincide con la columna de clave principal de la tabla SalesPerson. La columna SalesPersonID de la tabla SalesOrderHeader es la clave externa para la tabla SalesPerson.



No es necesario que una restricción FOREIGN KEY esté vinculada únicamente a una restricción PRIMARY KEY de otra tabla; también puede definirse para que haga referencia a las columnas de una restricción UNIQUE de otra tabla. Una restricción FOREIGN KEY puede contener valores NULL, pero si alguna columna de una restricción FOREIGN KEY compuesta contiene valores NULL, se omitirá la comprobación de los valores que componen la restricción FOREIGN KEY. Para asegurarse de que todos los valores de la restricción FOREIGN KEY compuesta se comprueben, especifique NOT NULL en todas las columnas que participan.

Si en una relación hay alguna clave ajena, sus valores deben coincidir con valores de la clave primaria a la que hace referencia, o bien, deben ser completamente nulos.

▮ Actualización y borrado de registros

La regla de integridad referencial se enmarca en términos de estados de la base de datos: indica lo que es un estado ilegal, pero no dice cómo puede evitarse. La cuestión es ¿qué hacer si estando en un estado legal, llega una petición para realizar una operación que conduce a un estado ilegal? Existen dos opciones: rechazar la operación, o bien aceptar la operación y realizar operaciones adicionales compensatorias que conduzcan a un estado legal.

Por lo tanto, para cada clave ajena de la base de datos habrá que contestar a tres preguntas:

Regla de los nulos: ¿Tiene sentido que la clave ajena acepte nulos?

Regla de borrado: ¿Qué ocurre si se intenta borrar la tupla referenciada por la clave ajena?

- Restringir: no se permite borrar la tupla referenciada.
- Propagar: se borra la tupla referenciada y se propaga el borrado a las tuplas que la referencian mediante la clave ajena.
- Anular: se borra la tupla referenciada y las tuplas que la referenciaban ponen a nulo la clave ajena (sólo si acepta nulos).

Regla de modificación: ¿Qué ocurre si se intenta modificar el valor de la clave primaria de la tupla referenciada por la clave ajena?

- Restringir: no se permite modificar el valor de la clave primaria de la tupla referenciada.
- Propagar: se modifica el valor de la clave primaria de la tupla referenciada y se propaga la modificación a las tuplas que la referencian mediante la clave ajena.
- Anular: se modifica la tupla referenciada y las tuplas que la referenciaban ponen a nulo la clave ajena (sólo si acepta nulos).

5. Otras reglas de integridad

Disparadores o triggers:

Combinan los enfoques declarativo (en la condición) y procedimental (en la acción), pueden ser tan complejas como imponga la semántica del mundo real en cuanto a la acción, y bastantes complejas en la condición (todo lo que permite la proposición lógica mediante la que se expresa la condición), El cumplimiento de la condición dispara la acción, Son más flexibles que las restricciones de acción específica.

Otro ejemplo de bases de datos

Siga los pasos dados en el ejemplo anterior

```
/* crear base de datos*/  
create database academica  
/*****/
```

```
/*acivar la base de datos*/
use academica
/*****/
create table profesor (
ced_pro int not null,/*valores requeridos*/
nom varchar (50) not null,
experiencia int not null,
ced_pro_c int not null,
/*clave principal*/
primary key (ced_pro),

/*relacion recursiva*/ foreign
key (ced_pro_c) references
profesor(ced_pro))

/*****/
create table depto (
cod_depto int not null,
nom_depto varchar (50) not
null, telefono int not null,
/*cuando la relacion es de uno a uno se crea un indice unico sobre el
campo
clave externa*/ ced_pro
int unique, primary key
(cod_depto),
/*relacion uno a muchos*/
foreign key (ced_pro)
references
profesor(ced_pro))

/*****/
create table programa (
cod_prog int not null,
nom_prog varchar (50) not
null, nivel varchar (50) not null,
cod_depto int not null,
/*regla de validacion*/ check (nom_prog in
('sistemas','quimica','industrial','mecanica','electronica','sa
nitaria')), primary key (cod_prog),
foreign key (cod_depto) references
```

```
depto(cod_depto))
/*****/
create table materia (
cod_ma int not null,
nom_ma varchar (50) not
null, n_horas int not null,
cod_prog int not null,
primary key (cod_ma),
foreign key (cod_prog)
references programa(cod_prog))
/*****/
create table estudiante (
carnet int not null,
nom_e varchar (50) not
null, tel int not null,
dir varchar (50) not null,
primary key (carnet))
/*****/
create table dicta (
ced_pro int not null,
cod_ma int not null,
/*esta tabal es una rompimiento o tabal de interseccion con clave
principal
compuesta y a la vez dos claves externas*/
primary key (ced_pro,cod_ma),
foreign key (ced_pro)
references profesor(ced_pro),
foreign key
(cod_ma)references
materia(cod_ma))
/*****/
create table matricula
( carnet int not null,
cod_ma int not null,
fecha_m datetime not null,
primary key (carnet,cod_ma),
foreign key (cod_ma)
references materia(cod_ma),
```

foreign key (carnet)
references
estudiante(carnet))

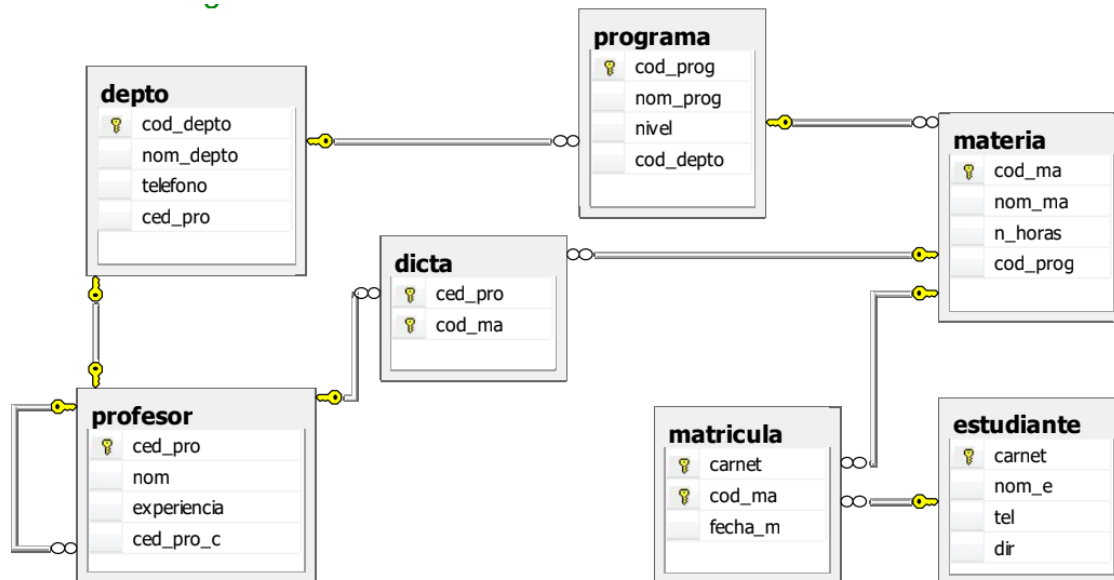
```
/******  
/*Insertar registros*/  
insert into profesor values  
(70325635,'alberto',12,70325635) insert into profesor  
values (56236859,'juaquin',22,70325635) insert into  
profesor values (10523698,'edilberto',15,70325635)  
insert into profesor values  
(41523698,'martha',30,70325635)  
  
select * from profesor  
/******  
insert into depto values  
(1,'electronica',2894145,70325635) insert into depto  
values (2,'Mecanica',2894145, 56236859) insert into  
depto values (3,'Diseño',2894145, 10523698) select  
* from depto  
/****** insert into  
programa values ('sistemas','bloque 1',1)  
insert into programa values ('quimica','bloque  
2',2) insert into programa values  
( 'industrial','bloque 3',3) insert into programa  
values ('mecanica','bloque 4',3) select * from  
programa  
/******  
insert into materia values (1111,'mantenimiento de pc',40,1)  
insert into materia values (2222,'creacion jabon',40,2)  
insert into materia values (3333,'el torno',40,3)  
insert into materia values (4444,'calibracion de valvulas',40,3)  
select * from materia  
/****** insert  
into dicta values (70325635,1111) insert  
into dicta values (56236859,2222) insert  
into dicta values (10523698,3333) insert  
into dicta values (41523698,4444) select  
* from dicta  
/******
```

```
insert into estudiante values (2008131001,'oscar',2895463,'cll 10')
```

```
insert into estudiante values
(2009132001,'cristian',4563125,'cll 50') insert into estudiante
values (2007133001,'andres',5453256,'kr 101') insert into
estudiante values (2008134001,'juan pablo',5423659,'kr 05')
select * from estudiante
```

```
/*******/
Insert      into      matricula      values
(2008131001,1111,getdate ()) insert into matricula
values (2009132001,2222,getdate ()) insert into
matricula values (2007133001,3333,getdate ())
insert      into      matricula      values
(2008134001,4444,getdate ()) select * from matricula
```

```
/*Este es el diagrama*/
```



Bibliografía.

Date, C.J. (1993). Introducción a los Sistemas de Bases de Datos. México: Addison-Wesley.

Hansen, G., & Hansen, J. (1997). Diseño y administración de Bases de Datos. México: Prentice

Hall. Korth, H., Silberschatz, A., & Sudarshan, S. (2006). Fundamentos de Bases de Datos. Madrid: McGraw-Hill.

Cibergrafía.

Introducción a SQL server 2008: Recuperado de:

[Introduccion_sql_server.pdf](#)

Recuperado de: <http://technet.microsoft.com/es-es/library/ms191236.aspx>

SQL tipo relaciones. Recuperado

de: <http://www.youtube.com/watch?v=rpAM-kldJg0&feature=related>

Reglas de integridad. Recuperado de:

<https://docs.microsoft.com/es-es/previous-versions/sql/sql-server-2008-r2/ms190765%28v%3dsql.105%29>

Proyecto de aprendizaje virtual. Recuperado de:

<http://issuu.com/ngiraldo/docs/basesdatos>

Tutorial de Bases de datos. . Recuperado de:

http://labredes.itcolima.edu.mx/fundamentosbd/sd_u2_3.htm



INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO®
UNIDAD DE EDUCACIÓN DIGITAL

